Java Methodenliste

Autor

Luis Staudt

20. Dezember 2025

Inhaltsverzeichnis

Inhaltsverzeichnis 2
Grundlegende Methoden in Java 3
String-Methoden 3
ConsoleReader - Eingaben von der Konsole 5
readString() - String einlesen 6
readInt() - Ganzzahl einlesen 7
readDouble() - Kommazahl einlesen 8
readByte() - Sehr kleine Ganzzahl einlesen 9
readShort() - Kleine Ganzzahl einlesen 10
readLong() - Sehr große Ganzzahl einlesen 11
readFloat() - Einfache Kommazahl einlesen 12
readIntArray() - Ganzzahl-Array einlesen 13
readDoubleArray() - Kommazahl-Array einlesen 14
readStringArray() - String-Array einlesen 15
Weitere Array-Methoden 17
Bildverarbeitung 18
show() - Bild anzeigen 19
loadFromFileSystem() - Bild von Festplatte laden 20
loadResource() - Bild aus Projekt laden 21
color() - Farbwert erstellen 23
red(), green(), blue() - Farbkanäle extrahieren 25
Häufige Fehler bei Strings 27

Grundlegende Methoden in Java

String-Methoden

length() - Länge eines Strings

Was macht die Methode?

Gibt die Anzahl der Zeichen im String zurück.

Syntax:

```
int laenge = text.length();
```

Beispiele:

```
String name = "Anna";
int laenge = name.length();  // laenge = 4

String wort = "Hallo Welt";
int laenge2 = wort.length();  // laenge2 = 10 (Leerzeichen zaehlt mit!)

String leer = "";
int laenge3 = leer.length();  // laenge3 = 0

String satz = "Java";
for (int i = 0; i < satz.length(); i++) {
    // Schleife laeuft 4 mal (von 0 bis 3)
}
```

Worauf achten:

- `length()` zählt alle Zeichen: Der Text "Hi"hat die Länge 2
- Leerzeichen und Sonderzeichen zählen mit
- Bei Strings: `.length()` mit Klammern
- Bei Arrays: `.length` ohne Klammern - das ist ein wichtiger Unterschied!

charAt(int index) - Zeichen an Position

Was macht die Methode?

Gibt dir das Zeichen an einer bestimmten Stelle im Text zurück. Wichtig: Die Zählung beginnt bei 0!

Syntax:

```
char zeichen = text.charAt(index);
```

Beispiele:

```
String wort = "Hallo";
char erstes = wort.charAt(0);   // erstes = 'H'
char zweites = wort.charAt(1);  // zweites = 'a'
char letztes = wort.charAt(4);  // letztes = 'o'

// Alle Zeichen durchlaufen
String text = "Java";
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    System.out.println(c);  // Gibt aus: J, a, v, a
}

// Pruefung einzelner Zeichen
String eingabe = "A1B2";
if (eingabe.charAt(0) == 'A') {
    System.out.println("Beginnt mit A");
}

// Zeichen vergleichen
String code = "12345";
if (code.charAt(2) == '3') {
    System.out.println("An Position 2 steht eine 3");
}
```

Worauf achten:

- Die Nummerierung beginnt bei 0
- Die Nummer muss zwischen 0 und `text.length() - 1` liegen
- Wenn die Nummer zu groß oder zu klein ist, stürzt das Programm ab
- Du bekommst ein einzelnes Zeichen (char) zurück, keinen Text (String)

ConsoleReader - Eingaben von der Konsole

Der `ConsoleReader` ist ein Werkzeug, mit dem du Eingaben vom Benutzer einlesen kannst. Um ihn zu benutzen, schreibst du am Anfang deines Programms:

```
import static prog.ConsoleReader.*;
```

Alle Eingabe-Methoden gibt es in zwei Varianten:

- Ohne Text: Wartet einfach auf die Eingabe
- Mit Text: Zeigt zuerst eine Aufforderung an (z.B. “Wie heißt du?“)

readString() - String einlesen

Was macht die Methode?

Liest eine Zeile Text von der Konsole ein und gibt sie als String zurück.

Syntax:

```
String text = readString();                // Ohne Prompt
String text = readString("Eingabetext");   // Mit Prompt
```

Beispiele:

```
// Einfache Eingabe
String name = readString("Wie heisst du? ");
System.out.println("Hallo " + name);

// Mehrere Eingaben
String vorname = readString("Vorname");
String nachname = readString("Nachname");
String vollername = vorname + " " + nachname;

// Ohne Prompt
System.out.println("Gib einen Text ein:");
String eingabe = readString();

// In Schleife verwenden
String antwort;
do {
    antwort = readString("Weiter? (ja/nein)");
} while (!antwort.equals("nein"));
```

Worauf achten:

- Liest alles, was der Benutzer eingibt, bis er Enter drückt
- Du bekommst immer Text (String) zurück, auch wenn jemand "123ëingibt
- Wenn du Zahlen brauchst, benutze `readInt()` oder `readDouble()`
- Wenn der Benutzer nur Enter drückt, bekommst du einen leeren Text zurück

readInt() - Ganzzahl einlesen

Was macht die Methode?

Liest eine Ganzzahl von der Konsole ein und gibt sie als `int` zurück.

Syntax:

```
int zahl = readInt();                        // Ohne Prompt
int zahl = readInt("Gib eine Zahl ein");     // Mit Prompt
```

Beispiele:

```
// Alter einlesen
int alter = readInt("Wie alt bist du? ");
System.out.println("Du bist " + alter + " Jahre alt");

// Berechnungen
int a = readInt("Erste Zahl");
int b = readInt("Zweite Zahl");
int summe = a + b;
System.out.println("Summe: " + summe);

// Array-Groesse einlesen
int anzahl = readInt("Wie viele Zahlen? ");
int[] zahlen = new int[anzahl];
for (int i = 0; i < zahlen.length; i++) {
    zahlen[i] = readInt("Zahl " + (i + 1));
}

// Schleifensteuerung
int wiederholungen = readInt("Wie oft wiederholen? ");
for (int i = 0; i < wiederholungen; i++) {
    System.out.println("Durchlauf " + (i + 1));
}
```

Worauf achten:

- Wenn jemand Text statt einer Zahl eingibt, stürzt das Programm ab
- Funktioniert nur für ganze Zahlen von etwa -2 Milliarden bis +2 Milliarden
- Für größere Zahlen: `readLong()` benutzen
- Für Kommazahlen: `readDouble()` benutzen

readDouble() - Kommazahl einlesen

Was macht die Methode?

Liest eine Kommazahl von der Konsole ein und gibt sie als `double` zurück.

Syntax:

```
double zahl = readDouble();                    // Ohne Prompt
double zahl = readDouble("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Preis einlesen
double preis = readDouble("Preis in Euro");
double mehrwertsteuer = preis * 0.19;
double gesamtpreis = preis + mehrwertsteuer;
System.out.println("Gesamt: " + gesamtpreis + " Euro");

// Durchschnitt berechnen
double note1 = readDouble("Note 1");
double note2 = readDouble("Note 2");
double note3 = readDouble("Note 3");
double durchschnitt = (note1 + note2 + note3) / 3.0;
System.out.println("Durchschnitt: " + durchschnitt);

// Temperatur
double celsius = readDouble("Temperatur in Celsius");
double fahrenheit = celsius * 9.0 / 5.0 + 32.0;
System.out.println(celsius + " C  = " + fahrenheit + " F ");

// Auch Ganzzahlen moeglich
double wert = readDouble("Zahl");  // Eingabe: 5
// wert = 5.0
```

Worauf achten:

- Je nach Computer funktioniert Komma `,` oder Punkt `.` als Trennzeichen
- Wenn jemand normalen Text eingibt, stürzt das Programm ab
- Du kannst auch ganze Zahlen eingeben - sie werden automatisch zu Kommazahlen
- Für die meisten Fälle ist `readDouble()` besser als `readFloat()`

readByte() - Sehr kleine Ganzzahl einlesen

Was macht die Methode?

Liest eine sehr kleine Ganzzahl von der Konsole ein und gibt sie als `byte` zurück.

Syntax:

```
byte zahl = readByte();                    // Ohne Prompt
byte zahl = readByte("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Alter (wenn < 127)
byte alter = readByte("Alter");

// Prozent (0-100)
byte prozent = readByte("Prozent (0-100)");
```

Worauf achten:

- Wertebereich: nur -128 bis 127
- Bei zu großen Zahlen: Fehler
- Meist wird `readInt()` bevorzugt

readShort() - Kleine Ganzzahl einlesen

Was macht die Methode?

Liest eine kleine Ganzzahl von der Konsole ein und gibt sie als `short` zurück.

Syntax:

```
short zahl = readShort();                    // Ohne Prompt
short zahl = readShort("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Jahr
short jahr = readShort("Geburtsjahr");

// Distanz in Metern
short meter = readShort("Distanz in Metern");
```

Worauf achten:

- Wertebereich: -32.768 bis 32.767
- Bei zu großen Zahlen: Fehler
- Meist wird `readInt()` bevorzugt

readLong() - Sehr große Ganzzahl einlesen

Was macht die Methode?

Liest eine sehr große Ganzzahl von der Konsole ein und gibt sie als `long` zurück.

Syntax:

```
long zahl = readLong();                    // Ohne Prompt
long zahl = readLong("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Sehr gro e Zahlen
long weltbevoelkerung = readLong("Weltbevoelkerung");

// Millisekunden
long millis = readLong("Millisekunden");
```

Worauf achten:

- Für sehr große Zahlen (größer als ca. 2 Milliarden)
- Wertebereich: ca. -9,2 $\times$ 10^{18} bis 9,2 $\times$ 10^{18}
- Für normale Zahlen: `readInt()` verwenden

readFloat() - Einfache Kommazahl einlesen

Was macht die Methode?

Liest eine Kommazahl mit einfacher Genauigkeit von der Konsole ein und gibt sie als `float` zurück.

Syntax:

```
float zahl = readFloat();                    // Ohne Prompt
float zahl = readFloat("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Gewicht
float gewicht = readFloat("Gewicht in kg");

// Groesse
float groesse = readFloat("Groesse in m");
float bmi = gewicht / (groesse * groesse);
```

Worauf achten:

- Weniger genau als `double` (ca. 6-7 Dezimalstellen)
- Meist wird `readDouble()` bevorzugt
- Dezimaltrennzeichen: Komma oder Punkt (je nach System)

readIntArray() - Ganzzahl-Array einlesen

Was macht die Methode?

Liest mehrere Ganzzahlen auf einmal ein und gibt sie als Array zurück. Die Zahlen müssen mit Leerzeichen getrennt werden.

Syntax:

```
int[] zahlen = readIntArray();                    // Ohne Prompt
int[] zahlen = readIntArray("Gib Zahlen ein");    // Mit Prompt
```

Beispiele:

```
// Mehrere Zahlen auf einmal einlesen
int[] noten = readIntArray("Noten (durch Leerzeichen getrennt)");
// Eingabe: 1 2 3 2 1
// noten = {1, 2, 3, 2, 1}

// Durchschnitt berechnen
int summe = 0;
for (int note : noten) {
    summe += note;
}
double durchschnitt = (double) summe / noten.length;
System.out.println("Durchschnitt: " + durchschnitt);

// Array direkt ausgeben
int[] zahlen = readIntArray("Zahlen");
System.out.println("Anzahl: " + zahlen.length);
for (int i = 0; i < zahlen.length; i++) {
    System.out.println("Zahl " + (i + 1) + ": " + zahlen[i]);
}
```

Worauf achten:

- Die Zahlen müssen mit Leerzeichen getrennt sein (z.B. "1 2 3“)
- Die Größe des Arrays passt sich automatisch an die Anzahl der Zahlen an
- Wenn jemand Text eingibt, stürzt das Programm ab
- Alle Zahlen müssen in einer Zeile stehen (Enter beendet die Eingabe)

readDoubleArray() - Kommazahl-Array einlesen

Was macht die Methode?

Liest mehrere Kommazahlen von der Konsole ein und gibt sie als `double[]`-Array zurück. Die Zahlen müssen durch Leerzeichen getrennt eingegeben werden.

Syntax:

```
double[] zahlen = readDoubleArray();                    // Ohne Prompt
double[] zahlen = readDoubleArray("Gib Zahlen ein");    // Mit Prompt
```

Beispiele:

```
// Preise einlesen
double[] preise = readDoubleArray("Preise");
// Eingabe: 9.99 14.50 7.25
// preise = {9.99, 14.50, 7.25}

// Gesamtpreis berechnen
double gesamt = 0.0;
for (double preis : preise) {
    gesamt += preis;
}
System.out.println("Gesamt: " + gesamt + " Euro");

// Temperaturwerte
double[] temps = readDoubleArray("Temperaturen");
double min = temps[0];
double max = temps[0];
for (double temp : temps) {
    if (temp < min) min = temp;
    if (temp > max) max = temp;
}
System.out.println("Min: " + min + ", Max: " + max);
```

Worauf achten:

- Zahlen müssen durch **Leerzeichen** getrennt werden
- Dezimaltrennzeichen: Komma oder Punkt (je nach System)
- Array-Größe ergibt sich automatisch aus der Anzahl der Eingaben
- Bei ungültiger Eingabe: Programm bricht ab

readStringArray() - String-Array einlesen

Was macht die Methode?

Liest mehrere Wörter von der Konsole ein und gibt sie als `String[]`-Array zurück. Die Wörter müssen durch Leerzeichen getrennt eingegeben werden.

Syntax:

```
String[] woerter = readStringArray();                    // Ohne Prompt
String[] woerter = readStringArray("Gib Woerter ein");   // Mit Prompt
```

Beispiele:

```
// Namen einlesen
String[] namen = readStringArray("Namen");
// Eingabe: Anna Bob Clara
// namen = {"Anna", "Bob", "Clara"}

// Alle Namen ausgeben
System.out.println("Teilnehmer:");
for (String name : namen) {
    System.out.println("- " + name);
}

// Woerter zaehlen
String[] woerter = readStringArray("Satz");
System.out.println("Anzahl Woerter: " + woerter.length);

// Laengstes Wort finden
String[] text = readStringArray("Text");
String laengstes = text[0];
for (String wort : text) {
    if (wort.length() > laengstes.length()) {
        laengstes = wort;
    }
}
System.out.println("Laengstes Wort: " + laengstes);
```

Worauf achten:

- Die Wörter müssen mit Leerzeichen getrennt sein
- Wenn du ganze Sätze einlesen willst, benutze lieber `readString()`
- Die Größe des Arrays passt sich automatisch an die Anzahl der Wörter an
- Für Sätze: `readString()` benutzen und dann mit `.split()` aufteilen

Weitere Array-Methoden

Der ConsoleReader bietet auch Methoden für andere Datentypen:

- `readByteArray()` - Liest `byte[]`-Array ein
- `readShortArray()` - Liest `short[]`-Array ein
- `readLongArray()` - Liest `long[]`-Array ein
- `readFloatArray()` - Liest `float[]`-Array ein

Diese funktionieren analog zu `readIntArray()` und `readDoubleArray()`.

Beispiel:

```
// Byte-Array
byte[] bytes = readByteArray("Bytes");

// Long-Array fuer grosse Zahlen
long[] ids = readLongArray("IDs");

// Float-Array
float[] gewichte = readFloatArray("Gewichte");
```

Bildverarbeitung

Die `Picture`-Klasse ist ein Werkzeug zum Arbeiten mit Bildern. Bilder werden als zweidimensionale Arrays gespeichert, wobei jeder Wert eine Farbe darstellt. Um sie zu benutzen, schreibst du am Anfang:

```
1 import static prog.Picture.*;
```

show() - Bild anzeigen

Was macht die Methode?

Zeigt ein Bild (als `int[][]`-Array) in einem Fenster an.

Syntax:

```
show(bildArray);                    // Ohne Fenstertitel
show(bildArray, "Fenstertitel");    // Mit Fenstertitel
```

Beispiele:

```
// Bild aus Datei laden und anzeigen
int[][] bild = loadFromFileSystem("pfad/zum/bild.jpg");
show(bild);

// Mit eigenem Fenstertitel
int[][] foto = loadFromFileSystem("foto.png");
show(foto, "Mein Urlaubsfoto");

// Mehrere Bilder anzeigen
int[][] bild1 = loadResource("bild1.jpg");
int[][] bild2 = loadResource("bild2.jpg");
show(bild1, "Original");
show(bild2, "Bearbeitet");

// Selbst erstelltes Bild anzeigen
int[][] neuBild = new int[100][100];
for (int y = 0; y < neuBild.length; y++) {
    for (int x = 0; x < neuBild[0].length; x++) {
        neuBild[y][x] = color(255, 0, 0);  // Rot
    }
}
show(neuBild, "Rotes Quadrat");
```

Worauf achten:

- Das Array hat das Format `int[Höhe][Breite]` - erst Höhe, dann Breite
- Jede Zahl im Array stellt eine Farbe dar
- Das Fenster bleibt offen, bis du es schließt
- Wenn du keinen Titel angibst, wird ein Standardtitel verwendet

loadFromFileSystem() - Bild von Festplatte laden

Was macht die Methode?

Lädt ein Bild von der Festplatte und gibt es als `int[][]`-Array zurück.

Syntax:

```
int[][] bild = loadFromFileSystem("pfad/zur/datei.jpg");
```

Beispiele:

```
// Absoluter Pfad
int[][] bild1 = loadFromFileSystem("/home/user/bilder/foto.jpg");
show(bild1);

// Relativer Pfad (relativ zum Projektverzeichnis)
int[][] bild2 = loadFromFileSystem("bilder/landschaft.png");
show(bild2, "Landschaft");

// Bild laden und Informationen ausgeben
int[][] foto = loadFromFileSystem("foto.jpg");
int hoehe = foto.length;
int breite = foto[0].length;
System.out.println("Bildgroesse: " + breite + " x " + hoehe);

// Bild laden und verarbeiten
int[][] original = loadFromFileSystem("bild.jpg");
int[][] bearbeitet = new int[original.length][original[0].length];
// ... Bildverarbeitung ...
show(original, "Original");
show(bearbeitet, "Bearbeitet");
```

Worauf achten:

- Funktioniert mit den gängigen Bildformaten: JPG, PNG, BMP, GIF
- Du kannst den vollständigen Pfad oder einen relativen Pfad angeben
- Wenn die Datei nicht existiert, stürzt das Programm ab
- Für Bilder, die zu deinem Programm gehören: benutze `loadResource()`

loadResource() - Bild aus Projekt laden

Was macht die Methode?

Lädt ein Bild, das zu deinem Projekt gehört, und gibt es als Array zurück. Das Bild muss im selben Ordner liegen wie dein Programm.

Syntax:

```
int[][] bild = loadResource("dateiname.jpg");
```

Beispiele:

```
// Bild direkt aus dem Package laden
int[][] logo = loadResource("logo.png");
show(logo, "Logo");

// Mehrere Bilder laden
int[][] bild1 = loadResource("bild1.jpg");
int[][] bild2 = loadResource("bild2.jpg");
show(bild1, "Bild 1");
show(bild2, "Bild 2");

// Bild laden und Groesse pruefen
int[][] foto = loadResource("foto.png");
if (foto.length > 0 && foto[0].length > 0) {
    System.out.println("Bild erfolgreich geladen");
    show(foto);
}

// Typische Verwendung in einfachen Projekten
// Dateistruktur:
// src/
//   prog/
//     Main.java
//     bild.jpg  <- Bild liegt direkt neben der Klasse

// Variante 1: Relativ
int[][] bild = loadResource("bild.jpg");
show(bild);

// Variante 2: Absolut vom Classpath-Root
int[][] bild2 = loadResource("/prog/bild.jpg");
show(bild2);
```

Worauf achten:

- Nur für Bilder benutzen, die zu deinem Projekt gehören
- Das Bild muss im selben Ordner liegen wie dein Programm
- Gib nur den Dateinamen an, keinen Pfad
- Für Bilder von woanders auf deinem Computer: benutze `loadFromFileSystem()`

color() - Farbwert erstellen

Was macht die Methode?

Erstellt einen RGB-Farbwert aus Rot-, Grün- und Blau-Anteilen.

Syntax:

```
int farbwert = color(rot, gruen, blau);  // Werte: 0-255
```

Beispiele:

```
// Grundfarben
int rot = color(255, 0, 0);
int gruen = color(0, 255, 0);
int blau = color(0, 0, 255);

// Mischfarben
int gelb = color(255, 255, 0);      // Rot + Gruen
int magenta = color(255, 0, 255);   // Rot + Blau
int cyan = color(0, 255, 255);      // Gruen + Blau

// Graustufen
int schwarz = color(0, 0, 0);
int weiss = color(255, 255, 255);
int grau = color(128, 128, 128);

// Bild mit Farbe fuellen
int[][] bild = new int[100][200];
int orange = color(255, 165, 0);
for (int y = 0; y < bild.length; y++) {
    for (int x = 0; x < bild[0].length; x++) {
        bild[y][x] = orange;
    }
}
show(bild, "Orange");

// Farbverlauf erstellen
int[][] verlauf = new int[256][100];
for (int y = 0; y < verlauf.length; y++) {
    int farbwert = color(y, 0, 255 - y);  // Von Blau zu Rot
    for (int x = 0; x < verlauf[0].length; x++) {
        verlauf[y][x] = farbwert;
    }
}
show(verlauf, "Farbverlauf");
```

```
35
36 // Pixel-Farbe setzen
37 int[][] foto = loadResource("foto.jpg");
38 foto[50][100] = color(255, 0, 0);  // Roten Punkt setzen
39 show(foto);
```

Worauf achten:

- Jede Farbe (Rot, Grün, Blau) kann einen Wert von 0 bis 255 haben
- 0 bedeutet "gar nichts von dieser Farbe", 255 bedeutet mmaximale Helligkeit"
- RGB steht für: Rot, Grün, Blau
- Die Methode gibt eine einzige Zahl zurück, die alle drei Farben enthält

red(), green(), blue() - Farbkanäle extrahieren

Was machen die Methoden?

Holen den Rot-, Grün- oder Blau-Anteil aus einer Farbe heraus.

Syntax:

```
int rotWert = red(farbwert);      // Rot-Anteil (0-255)
int gruenWert = green(farbwert);  // Gruen-Anteil (0-255)
int blauWert = blue(farbwert);    // Blau-Anteil (0-255)
```

Beispiele:

```
// Farbkan le eines Farbwerts extrahieren
int orange = color(255, 165, 0);
int r = red(orange);      // r = 255
int g = green(orange);    // g = 165
int b = blue(orange);     // b = 0
System.out.println("RGB: " + r + ", " + g + ", " + b);

// Pixel-Farbe analysieren
int[][] bild = loadResource("foto.jpg");
int pixel = bild[50][100];  // Pixel an Position (100, 50)
System.out.println("Rot: " + red(pixel));
System.out.println("Gruen: " + green(pixel));
System.out.println("Blau: " + blue(pixel));

// Bild in Graustufen umwandeln
int[][] original = loadResource("bild.jpg");
int[][] graustufen = new int[original.length][original[0].length];
for (int y = 0; y < original.length; y++) {
    for (int x = 0; x < original[0].length; x++) {
        int pixel = original[y][x];
        int r = red(pixel);
        int g = green(pixel);
        int b = blue(pixel);
        int grau = (r + g + b) / 3;  // Durchschnitt
        graustufen[y][x] = color(grau, grau, grau);
    }
}
show(original, "Original");
show(graustufen, "Graustufen");

// Nur Rot-Kanal behalten
int[][] nurRot = new int[original.length][original[0].length];
```

```
for (int y = 0; y < original.length; y++) {
    for (int x = 0; x < original[0].length; x++) {
        int r = red(original[y][x]);
        nurRot[y][x] = color(r, 0, 0);
    }
}
show(nurRot, "Nur Rot-Kanal");

// Farbkan le vertauschen (Rot <-> Blau)
int[][] vertauscht = new int[original.length][original[0].length];
for (int y = 0; y < original.length; y++) {
    for (int x = 0; x < original[0].length; x++) {
        int pixel = original[y][x];
        int r = red(pixel);
        int g = green(pixel);
        int b = blue(pixel);
        vertauscht[y][x] = color(b, g, r);  // Blau und Rot tauschen
    }
}
show(vertauscht, "Vertauschte Kan le");

// Helligkeit eines Pixels berechnen
int pixel = bild[0][0];
int helligkeit = (red(pixel) + green(pixel) + blue(pixel)) / 3;
System.out.println("Helligkeit: " + helligkeit);
```

Worauf achten:

- Du bekommst immer eine Zahl zwischen 0 und 255 zurück
- Sehr nützlich, wenn du Bilder bearbeiten willst
- Zusammen mit `color()` kannst du einzelne Farbanteile ändern
- Häufige Anwendung: Bilder in Graustufen umwandeln oder Farbfilter erstellen

Häufige Fehler bei Strings

String.length ohne Klammern

```
// FALSCH
String text = "Hallo";
int laenge = text.length;  // Fehler: Strings brauchen ()

// RICHTIG
int laenge = text.length();    // String MIT Klammern
```

Index-Grenzen bei charAt

```
String wort = "Test";  // length = 4, Indizes: 0, 1, 2, 3
char c = wort.charAt(4);  // Fehler! Index 4 existiert nicht

// RICHTIG
char letztes = wort.charAt(wort.length() - 1);  // Index 3
char erstes = wort.charAt(0);                   // Index 0
```

char vs. String

```
String text = "A";
char buchstabe = 'A';

// FALSCH
if (text.charAt(0) == "A") { }  // Vergleicht char mit String

// RICHTIG
if (text.charAt(0) == 'A') { }  // char mit char vergleichen
```